CREAR NUESTRA PRIMERA API EN PHP

INTRODUCCIÓN

Vamos a crear una API de consulta de usuarios utilizando una base de datos MySQL que ya tenemos y una estructura MVC sencilla.

Para que la explicación sea progresiva, empezaremos por una operación: consultar todos los usuarios y devolverlos en JSON. Después ampliaremos la API para consultar un usuario concreto, crear, modificar y eliminar.

PASO 1. PREPARAR LA BASE DE DATOS

Podemos utilizar la tabla usuarios que se proporciona. Para esta práctica supondremos que tiene estos campos:

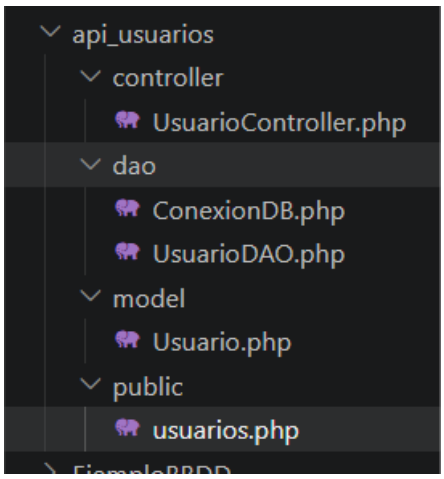
```
CREATE DATABASE api_dwes
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE api_dwes;

CREATE TABLE usuarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE
);

INSERT INTO usuarios (nombre, email) VALUES
('Ane', 'ane@ejemplo.com'),
('Jon', 'jon@ejemplo.com'),
('Miren', 'miren@ejemplo.com');
```

PASO 2. ORGANIZAR EL PROYECTO



En este ejemplo, **public/usuarios.php** será el punto de entrada HTTP de la API. Recibirá la petición, llamará al controlador y devolverá JSON.

PASO 3. CREAR EL MODELO USUARIO.PHP

El modelo representa la información del usuario. Seguimos trabajando con MVC.

```php
<?php
    namespace model;

    class Usuario
    {
        private int $id;
        private string $nombre;
        private string $email;

        public function __construct(
            int $id,
            string $nombre,
            string $email
        ) {
            $this->id = $id;
            $this->nombre = $nombre;
            $this->email = $email;
        }

        public function getId(): int
        {
            return $this->id;
        }

        public function getNombre(): string
        {
            return $this->nombre;
        }

        public function getEmail(): string
        {
            return $this->email;
        }
    }
?>
```

La API no cambia el modelo. Seguimos representando a los usuarios como objetos PHP.

PASO 4. CREAR LA CONEXIÓN CONEXIONDB.PHP

Utilizar una conexión anterior modificando el nombre de la BD.

```php
<?php
    class ConexionDB
    {
        public static function conexion(): mysqli
        {
            $conexion = new mysqli(
                "localhost",
                "root",
                "",
                "api_dwes"
            );
            // Indicamos que utilizaremos UTF-8.
            // Permite trabajar correctamente con caracteres como á, é, ñ...
            $conexion->query("SET NAMES 'utf8'");
            // Devolvemos la conexión.
            return $conexion;
        }
    }
?>
```

Para una aplicación real, la conexión debe manejar correctamente los errores y utilizar credenciales configuradas de forma segura.

PASO 5. CREAR EL DAO USUARIODAO.PHP

El DAO es el responsable de acceder a la base de datos. No debe saber si los datos se van a mostrar en HTML o se van a devolver en JSON.

```php
<?php

    namespace dao;

    use model\Usuario;
    use mysqli;
    use \ConexionDB;

    class UsuarioDAO
    {
        private mysqli $conexion;
        public function __construct()
        {
            $this->conexion = ConexionDB::conexion();
        }
        public function buscarTodos(): array
        {
            $sql = "SELECT id, nombre, email
                    FROM usuarios
                    ORDER BY id";

            $resultado = $this->conexion->query($sql);
            $usuarios = [];
            while ($fila = $resultado->fetch_assoc()) {
                $usuarios[] = new Usuario(
                    (int) $fila["id"],
                    $fila["nombre"],
                    $fila["email"]
                );
            }
            return $usuarios;
        }
    }
?>
```

Fíjate en que **buscarTodos()** devuelve un array de objetos Usuario. No genera HTML y tampoco convierte los datos a JSON.

¿Por qué? Porque su responsabilidad es recuperar los datos de la base de datos. La capa que prepara la respuesta HTTP será la encargada de decidir cómo se envían.

PASO 6. CREAR EL CONTROLADOR USUARIOCONTROLLER.PHP

El controlador coordina la operación y transforma los objetos del modelo en una estructura que se puede serializar a JSON.

```php
<?php
    namespace controller;

    use dao\UsuarioDAO;

    class UsuarioController
    {
        private UsuarioDAO $usuarioDAO;

        public function __construct()
        {
            $this->usuarioDAO = new UsuarioDAO();
        }

        public function listar(): array
        {
            $usuarios = $this->usuarioDAO->buscarTodos();

            $datos = [];

            foreach ($usuarios as $usuario) {
                $datos[] = [
                    "id" => $usuario->getId(),
                    "nombre" => $usuario->getNombre(),
                    "email" => $usuario->getEmail()
                ];
            }

            return $datos;
        }
    }
?>
```

Aquí aparece una diferencia importante respecto a una vista HTML: el controlador devuelve un array de datos, preparado para convertirse en JSON.

PASO 7. CREAR EL PUNTO DE ENTRADA DE LA API

Ahora vamos a crear **public/usuarios.php**. Este archivo es el que recibirá la petición del cliente y devolverá los datos. Es importante distinguirlo del controlador: el punto de entrada gestiona la comunicación HTTP, mientras que el controlador coordina la lógica de la aplicación.

```php
//ejecutar:http://localhost:8000/UD03/api_usuarios/public/usuarios.php
<?php
    // Cargar las clases necesarias
    require_once("../model/Usuario.php");
    require_once("../dao/ConexionDB.php");
    require_once("../dao/UsuarioDAO.php");
    require_once("../controller/UsuarioController.php");
    //Importar el controlador
    use controller\UsuarioController;

    // Indicamos que la respuesta será JSON.
    header("Content-Type: application/json; charset=utf-8");
    //Informa al cliente de que la respuesta es JSON codificado en UTF-8.

    // Permite comprobar qué método HTTP ha utilizado el cliente.
    if ($_SERVER["REQUEST_METHOD"] !== "GET") {
        //comunica que el método no está permitido.
        http_response_code(405);

        echo json_encode([
            "error" => "Método HTTP no permitido"
        ]);

        exit;
    }

    // Creamos el controlador.
    $controller = new UsuarioController();

    // Reutiliza el controlador y el DAO que acabamos de implementar.
    $usuarios = $controller->listar();

    // json_encod: Convierte el array PHP a JSON.
    echo json_encode(
        $usuarios,
        JSON_UNESCAPED_UNICODE | JSON_PRETTY_PRINT
    );
    // Envía el contenido al cliente en el cuerpo de la respuesta HTTP.
?>
```

En los archivos de este ejemplo se utilizan rutas relativas sencillas, como en las prácticas iniciales. Es necesario ejecutarlo a través del servidor web y asegurarse de que las rutas de los **require_once** se resuelven desde la ubicación del archivo de entrada.

JSON_UNESCAPED_UNICODE

Esta opción evita que los caracteres Unicode, como ñ o las vocales acentuadas, se representen mediante secuencias de escape cuando se pueden escribir directamente.

JSON_PRETTY_PRINT

Esta opción añade saltos de línea y espacios para que el JSON sea más fácil de leer. Sin esta opción, el resultado podría aparecer en una sola línea:

Una API no consiste únicamente en llamar a **json_encode().** También debemos gestionar la petición HTTP y devolver una respuesta coherente.

PASO 8. PROBAR LA API

Arrancamos Apache y MySQL en XAMPP y accedemos desde el navegador a la dirección correspondiente. Por ejemplo, si el proyecto está en \htdocs\DSER\Public\UD03\api_usuarios

http://localhost:8000/UD03/api_usuarios/public/usuarios.php


```
[
    {
        "id": 1,
        "nombre": "Ane",
        "email": "ane@ejemplo.com"
    },
    {
        "id": 2,
        "nombre": "Jon",
        "email": "jon@ejemplo.com"
    },
    {
        "id": 3,
        "nombre": "Miren",
        "email": "miren@ejemplo.com"
    }
]
```


El navegador muestra el JSON directamente, sin una tabla HTML, sin etiquetas table, tr o td y sin una plantilla de PHP.

Si añadimos un cuarto usuario a la base de datos, la API lo devolverá automáticamente, porque los datos proceden de la consulta SQL.

PASO 9. CONSULTAR UN USUARIO POR SU ID

El objetivo es poder hacer una petición como esta:

http://localhost:8000/UD03/api_usuarios/public/usuarios.php?id=2

La API devolverá los datos del usuario cuyo ID sea 2, en formato JSON.

Vamos a modificar únicamente los archivos necesarios para añadir esta funcionalidad.

9.1. MODIFICAR DAO/USUARIODAO.PHP

Añade el método **buscarPorId()** dentro de la clase **UsuarioDAO**, debajo del método **buscarTodos().**

```php
public function buscarPorId(int $id): ?Usuario
{
    $sql = "SELECT id, nombre, email
            FROM usuarios
            WHERE id = ?";

    $stmt = $this->conexion->prepare($sql);
    $stmt->bind_param("i", $id);
    $stmt->execute();

    $resultado = $stmt->get_result();

    if ($fila = $resultado->fetch_assoc()) {
        return new Usuario(
            (int) $fila["id"],
            $fila["nombre"],
            $fila["email"]
        );
    }
    return null;
}
```

9.2. MODIFICAR CONTROLLER/USUARIOCONTROLLER.PHP

Añade este método debajo de **listar(),** sin eliminar el método que ya funciona.

```php
public function buscarPorId(int $id): ?array
{
    $usuario = $this->usuarioDAO->buscarPorId($id);

    if ($usuario === null) {
        return null;
    }

    return [
        "id" => $usuario->getId(),
        "nombre" => $usuario->getNombre(),
        "email" => $usuario->getEmail()
    ];
}
```

El controlador recibe el ID, llama al DAO y transforma el objeto Usuario en un array que después convertiremos a JSON.

9.3. MODIFICAR PUBLIC/USUARIOS.PHP

Ahora tenemos que distinguir dos peticiones:

- Sin parámetro id: consultar todos los usuarios, como hasta ahora.
- Con parámetro id: consultar un único usuario.

Mantén los require_once que ya tienes y sustituye el bloque que comprueba el método HTTP y ejecuta la consulta por este código:

```php
<?php
// Indicamos que la respuesta será JSON.
header("Content-Type: application/json; charset=utf-8");

// Solo permitimos peticiones GET.
if ($_SERVER["REQUEST_METHOD"] !== "GET") {
    http_response_code(405);

    echo json_encode([
        "error" => "Método HTTP no permitido"
    ]);

    exit;
}
```

```php
// Creamos el controlador.
$controller = new \controller\UsuarioController();

// Comprobamos si se ha recibido un ID.
if (isset($_GET["id"])) {
    $id = filter_var($_GET["id"], FILTER_VALIDATE_INT);

    if ($id === false || $id === null || $id <= 0) {
        http_response_code(400);

        echo json_encode([
            "error" => "El ID debe ser un entero positivo"
        ]);

        exit;
    }

    // Buscamos el usuario por su ID.
    $usuario = $controller->buscarPorId($id);

    if ($usuario === null) {
        http_response_code(404);

        echo json_encode([
            "error" => "Usuario no encontrado"
        ]);

        exit;
    }

    echo json_encode(
        $usuario,
        JSON_UNESCAPED_UNICODE | JSON_PRETTY_PRINT
    );
```

```php
} else {
    // Si no hay ID, mostramos todos los usuarios.
    $usuarios = $controller->listar();

    echo json_encode(
        $usuarios,
        JSON_UNESCAPED_UNICODE | JSON_PRETTY_PRINT
    );
}
?>
```

Ahora tenemos tres posibles resultados:

- Consultar todos los usuarios **200** **Array JSON**
- Consultar un ID inexistente **404** **Objeto JSON con error**
- Enviar un ID inválido **400** **Objeto JSON con error**

CÓMO COMPLETAR EL CRUD DE UNA API

El siguiente objetivo sería permitir que otra aplicación cree, modifique y elimine usuarios. El esquema sería el siguiente:

- **Consultar usuarios:**
  - **Metodo:** GET
  - **Datos de entrada:** Parámetros de consulta, si hacen falta
  - **Codigo habitual**: 200
- **Crear usuario**
  - **Metodo:** POST
  - **Datos de entrada:** Cuerpo JSON
  - **Codigo habitual**: 201
- **Actualizar usuario**
  - **Metodo:** PUT
  - **Datos de entrada:** ID y cuerpo JSON
  - **Codigo habitual**: 200 o 204
- **Eliminar usuario**
  - **Metodo:** DELETE
  - **Datos de entrada:** ID
  - **Codigo habitual**: 204

Para implementar las operaciones restantes, aprovechamos los métodos del DAO que ya conocen: **insertar(), modificar()** y **eliminar().**

La novedad es que los datos de entrada y salida se intercambian mediante HTTP y JSON.

CREAR UN USUARIO MEDIANTE POST

El cliente podría enviar una petición con este cuerpo JSON:

```
{
    "nombre": "Nerea",
    "email": "nerea@gmail.com"
}
```

En PHP podemos leer el cuerpo de la petición con **file_get_contents("php://input")** y convertirlo a un array asociativo mediante **json_decode().**

```
$contenido = file_get_contents("php://input");
$datos = json_decode($contenido, true);
if (
    !is_array($datos) ||
    !isset($datos["nombre"], $datos["email"]) ||
    !is_string($datos["nombre"]) ||
    !is_string($datos["email"]) ||
    trim($datos["nombre"]) === "" ||
    !filter_var($datos["email"], FILTER_VALIDATE_EMAIL)
) {
    http_response_code(400);
    echo json_encode([
        "error" => "Los datos enviados no son válidos"
    ]);
    exit;
}
```

Este bloque valida la estructura y algunos campos del JSON recibido. Después, el controlador puede utilizar el DAO para insertar el usuario en MySQL.

Conviene explicar que json_decode() no valida automáticamente los datos de negocio: tenemos que comprobar los campos obligatorios, sus tipos, sus formatos y las restricciones de la base de datos.

Además, debemos evitar que se creen usuarios duplicados si el correo tiene una restricción UNIQUE, y gestionar los errores de la operación.

BUENAS PRACTICAS

- **Separación de capas:** el DAO no genera JSON ni HTML; se ocupa del acceso a los datos.
- **Validación:** no podemos confiar en los datos que envía el cliente.
- **Códigos HTTP:** utilizar 200, 201, 204, 400, 401, 403, 404 y 500 según corresponda.
- **Errores en JSON:** las respuestas de error también deben tener un formato coherente.
- **Seguridad:** las operaciones que modifican datos deben tener los controles de autenticación y autorización necesarios.
- **Consultas preparadas:** utilizar parámetros en SQL para evitar inyecciones SQL.
- **Datos sensibles:** no devolver contraseñas, hashes de contraseñas ni información que el cliente no deba conocer.
- **Configuración:** no mostrar errores internos, contraseñas de la base de datos ni trazas de ejecución a los clientes.